



**T.C.
FATİH SULTAN MEHMET VAKIF UNIVERSITY**

**Faculty of Engineering Department
of Software Engineering**

2025-2026 Academic Year

Parallel Programming Final Project Report

Student Name: Zeynep Dağtekin

Student ID: 2121251007

Course: Parallel Programming

Course Instructor: Asst. Prof. Dr. Ümit Demirbağa

Project Title: Parallel Image Processing Pipeline with OpenMP

Programming Language: C

Project Language: English

Submission Date: 06 June 2026

Parallel Image Processing Pipeline with OpenMP

1. Abstract

I built an image processing pipeline in C and parallelized it using OpenMP. This pipeline has four stages: converting a synthetic RGB image to grayscale, applying a 3x3 blur filter, detecting edges with Sobel method and thresholding to get a black and white output. I first wrote a sequential version then a parallel one where i split pixel loops across multiple threads using OpenMP work sharing constructs.

I tested it on synthetic images of three different sizes: 1024x1024, 2048x2048 and 4096x4096. Each test was repeated five times and I recorded the average execution time. I also compared static, dynamic and guided scheduling strategies and to verify correctness i used checksums and compared output images pixel by pixel between sequential and parallel versions. In every test number of mismatched pixels was zero so both versions produced identical results.

2. Introduction

Parallel programming, which breaks a task down into smaller components and executes them simultaneously reduces execution time. While sequential programs perform operations step by step using a single thread parallel programs distribute the work across multiple threads. Image processing is actually great choice for parallel programming because images are simply large pixel matrices and most operations perform nearly same operation on every pixel. Typically each output pixel is calculated independently or requires only a few neighboring pixels as input so it can be easily parallelized using loops.

3. Description of Selected Problem

Core problem is take a generated RGB image process it through several stages and create an output image showing edges. I opted to generate the input image in program itself not read it from a file mainly to keep experiments repeatable and straightforward. Every run gets same input size and pattern and pipeline goes like this:

RGB Image -Grayscale - 3x3 Blur -Sobel Edge Detection -Thresholding -Output Image

This is what happens in each step:

- **RGB to Grayscale:** Combine the red, green, and blue values into a single gray value
- **3x3 Blur:** Smooth every pixel using the average of its 3x3 neighborhood
- **Sobel Edge Detection:** To identify edges compute variations in intensity both horizontally and vertically
- **Thresholding:** When a pixel's edge value exceeds a predetermined threshold it becomes white otherwise it remains black

This whole process is parallelizable because each output pixel has its own memory location. In blur and Sobel stages threads only read neighboring pixels but write into separate buffers so there is no risk of threads conflicting with each other

4. Sequential Solution

In the sequential solution all pipeline stages are executed by one thread. Each stage scans image using two nested loops and outer loop usually goes over rows and inner loop goes over columns. For 4096 x 4096 image means that more than sixteen million pixels are processed in every stage

Main flow of sequential algorithm is:

1. Create synthetic RGB image
2. Convert every pixel from RGB to grayscale
3. Apply a 3x3 blur filter to grayscale image
4. Apply Sobel edge detection to blurred image
5. Apply thresholding and calculate a checksum
6. Write final output as a PGM file

blur and Sobel parts are more expensive than grayscale and thresholding because they read neighboring pixels for every pixel. Sequential result is used as baseline. Sequential and parallel execution times are compared to determine speedup

5. Parallel Solution with OpenMP

In parallel solution pixel loops on image were divided between threads by using OpenMP. In code one parallel region was opened and each pipeline step was shared with omp for. This approach reduces cost of creating threads again and again for each step

Main OpenMP structures used in project are as follows

OpenMP Structure	Purpose in Project
<code>#pragma omp parallel</code>	Used to start a parallel region with threads.
<code>#pragma omp for</code>	Divides pixel loops between threads.
<code>collapse(2)</code>	Combines nested y and x loops into one larger iteration space.
<code>schedule(runtime)</code>	Allows scheduling type to be selected at runtime.
<code>omp_set_schedule()</code>	Used to run static, dynamic and guided scheduling experiments.
<code>default(none)</code>	Makes it necessary to clearly write the scope of each variable.
<code>shared</code>	image buffers are shared because all threads access same arrays but each thread writes to a different output index
<code>private</code>	Makes loop variables such as x and y separate for each thread.
<code>firstprivate</code>	Copies width, height and threshold values to each thread.
<code>reduction(+:checksum)</code>	Combines partial checksum values safely without a race condition.

There is a dependency between pipeline stages. For example blur stage must wait until grayscale stage finishes and Sobel stage must wait until blur stage finishes. OpenMP for directive has an implicit barrier at the end by default. I didn't use `nowait` because starting next stage early could make a thread read incomplete data.

6. Code Blocks and Explanations

In this section i explain important parts of code block by block and i didn't explain every single line because full source code is already given in appendix. Instead of i focused on parts that are important for algorithm, OpenMP usage, correctness and performance

6.1 Libraries and Constants

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <omp.h>

#define THRESHOLD 100
#define CHUNK_SIZE 128
```

omp.h header is required for OpenMP directives and timing functions such as `omp_get_wtime()`.

THRESHOLD value is used in last stage of pipeline to decide whether an edge pixel becomes black or white. CHUNK_SIZE is used when selecting runtime scheduling policies. It is simpler to modify and explain code when these values are kept as

constants

6.2 Synthetic RGB Image Generation

```
static void generate_rgb_image(unsigned char *r, unsigned char *g, unsigned char *b,
                              int width, int height) {
    for (int y = 0; y < height; y++) {
        for (int x = 0; x < width; x++) {
            int idx = y * width + x;

            r[idx] = (unsigned char)((x * 255) / width);
            g[idx] = (unsigned char)((y * 255) / height);
            b[idx] = (unsigned char)((((x / 32) + (y / 32)) % 2) * 255);
        }
    }
}
```

This function creates input image inside program. For the red, green and blue channels image is saved as three distinct 1D arrays. Expression `idx = y * width + x` converts a 2D pixel position into a one dimensional array index and this makes memory allocation simpler in C

Generating image inside program also makes tests easier to repeat. If an external image file were used file reading time or file differences could affect experiments. In this project timing mainly focuses on image processing pipeline itself

6.3 Sequential Grayscale Conversion

```
for (int y = 0; y < height; y++) {
    for (int x = 0; x < width; x++) {
        int idx = y * width + x;
        gray[idx] = (unsigned char)((77 * r[idx] + 150 * g[idx] + 29 * b[idx]) / 256);
    }
}
```

This is grayscale step in sequential version. Each RGB pixel is converted into one gray value. Integer approximation of $0.299R + 0.587G + 0.114B$ is formula because integer arithmetic is straightforward and deterministic i chose it over floating point arithmetic. This also helps sequential and parallel results match exactly

6.4 OpenMP Parallel Region and Variable Scope

```
#pragma omp parallel default(none) shared(r, g, b, gray, blur, edge, out)
```

```
firstprivate(width, height, threshold) private(x, y) reduction(+:checksum)...
```

Block is main parallel region and all threads are created once and then reused for pipeline stages. I used default(none) because it forces all variables to be declared as shared, private, firstprivate or reduction. This is useful for avoiding hidden mistakes in variable scope

Image buffers are shared because all threads access same input and output arrays. In contrast loop design makes each thread write to a different pixel index and variables x and y are private because each thread needs its own loop variables. Checksum is handled with reduction because many threads add to same total value

6.5 Parallel Loop with collapse(2) and schedule(runtime)

```
#pragma omp for collapse(2) schedule(runtime)
for (y = 0; y < height; y++) {
    for (x = 0; x < width; x++) {
        int idx = y * width + x;
        gray[idx] = (unsigned char)((77 * r[idx] + 150 * g[idx] + 29 * b[idx]) / 256);
    }
}
```

The omp for directive is what actually hands out work to threads it takes the loop iterations and distributes them so each thread gets its own chunk to process independently

The collapse(2) clause is a smart method that combines the row loop and column loop into a single, large, flat pool of iterations rather than parallelizing only the outer loop. Imagine that a grid is unrolled into a single, lengthy list, which is then divided into threads

This pairs especially well with image processing. Images are made of potentially millions of pixels and since each pixel can usually be processed on its own there is a natural opportunity to divide the work at the finest possible level one pixel at a time keeping every thread busy with minimal idle time

6.6 Blur Stage and Buffer Separation

```
for (int ky = -1; ky <= 1; ky++) {  
    for (int kx = -1; kx <= 1; kx++) {  
        sum += gray[(y + ky) * width + (x + kx)];  
    }  
}  
  
blur[idx] = (unsigned char)(sum / 9);
```

Blur stage uses 3x3 neighborhood of a pixel. It reads values from gray buffer and writes result to blur buffer and this separation is important. If same array were used for reading and writing a thread could read a pixel that had already been changed by another thread. That would make result depend on execution order. By using separate input and output buffers this data dependency risk is controlled

6.7 Sobel Edge Detection

```
gx += -1 * blur[(y - 1) * width + (x - 1)];  
gx += 1 * blur[(y - 1) * width + (x + 1)];  
gx += -2 * blur[y * width + (x - 1)];  
gx += 2 * blur[y * width + (x + 1)];  
gx += -1 * blur[(y + 1) * width + (x - 1)];  
gx += 1 * blur[(y + 1) * width + (x + 1)];  
  
gy += -1 * blur[(y - 1) * width + (x - 1)];  
gy += -2 * blur[(y - 1) * width + x];  
gy += -1 * blur[(y - 1) * width + (x + 1)];  
gy += 1 * blur[(y + 1) * width + (x - 1)];  
gy += 2 * blur[(y + 1) * width + x];  
gy += 1 * blur[(y + 1) * width + (x + 1)];  
  
edge[idx] = clamp_int(abs(gx) + abs(gy));
```

Sobel stage uses gx to evaluate changes in horizontal intensity and gy to estimate changes in vertical intensity. If change is large pixel is likely to be on an edge. Result is written to edge buffer. Similar to blur stage, input buffer and output buffer are different so each thread can safely calculate a separate output pixel

6.8 Thresholding and Checksum Reduction

```
#pragma omp for collapse(2) schedule(runtime)
for (y = 0; y < height; y++) {
    for (x = 0; x < width; x++) {
        int idx = y * width + x;

        out[idx] = (edge[idx] > threshold) ? 255 : 0;
        checksum += out[idx];
    }
}
```

In this stage edge image is converted into a black and white image. Checksum is also calculated in same loop. Since multiple threads update checksum value a normal shared variable would create a race condition. For this reason, reduction(+:checksum) is used in parallel region. Each thread first computes its own partial sum and OpenMP combines these sums safely at the end

6.9 Runtime Scheduling Selection

```
static void set_schedule_policy(const char *schedule_name) {
    if (strcmp(schedule_name, "static") == 0) {
        omp_set_schedule(omp_sched_static, CHUNK_SIZE);
    } else if (strcmp(schedule_name, "dynamic") == 0) {
        omp_set_schedule(omp_sched_dynamic, CHUNK_SIZE);
    } else if (strcmp(schedule_name, "guided") == 0) {
        omp_set_schedule(omp_sched_guided, CHUNK_SIZE);
    }
}
```

This function chooses how work gets distributed across threads. Static scheduling divides iterations upfront in a fixed way. Dynamic gives threads new chunks on the fly as they finish which helps when iterations vary in cost. Guided starts with large chunks then shrinks them toward the end

The comparison shows how each policy affects pipeline performance because the best choice really depends on the workload. Runtime isn't its own scheduling strategy. It is only method of changing between static, dynamic or guided code during execution without having to recompile the code

6.10 Execution Time Measurement and Correctness Check

```
for (int i = 0; i < runs; i++) {  
    double start = omp_get_wtime();  
  
    par_checksum = parallel_pipeline(r, g, b,  
                                   gray_par, blur_par, edge_par, out_par,  
                                   width, height, THRESHOLD);  
  
    double end = omp_get_wtime();  
    par_total_time += end - start;  
}
```

Execution time was measured using the `omp_get_wtime()` function. Every experiment was carried out five times and the average result was recorded. After running sequential and parallel versions, program compares two output images. In my results mismatched pixel count was always 0. This shows that parallelization didn't change output

7. Data Dependency, Race Condition, and Synchronization Analysis

In image processing pipeline main dependency is between steps. Grayscale output is input of blur step. Blur output is input of Sobel step. Sobel output is input of thresholding step and because of this the stages cannot be executed in any order

To avoid incorrect results, I used separate buffers for each stage: gray, blur, edge and out. Blur stage for instance, reads from gray and writes to blur. Sobel stage writes to edge and reads from blur and that means that one thread doesn't read a value while another thread is updating same location

The only shared accumulation in program is checksum. This value is used to help verify output. If all threads updated checksum directly final value could be wrong because of a race condition. I used OpenMP reduction for checksum instead of critical or atomic. Critical and atomic weren't necessary for pixel writes because each output pixel has a separate index. Reduction is also cleaner for summation than protecting every addition with a critical section

Implicit barrier at end of each `omp for` loop is also important. It makes sure that one pipeline stage is fully completed before next one begins. I didn't use `nowait` because it could allow a later stage to start before previous buffer was completely ready

Some OpenMP synchronization and variable scope constructs were not used because they weren't necessary for this problem. critical and atomic weren't used for pixel writes because each thread writes to a different output index. For checksum calculation, reduction was preferred because it is more suitable for summation. master wasn't needed because there was no operation that only one thread had to perform inside the parallel region. ordered wasn't used because pixel processing doesn't require preserving loop order. flush wasn't used explicitly because implicit barriers between pipeline stages already provide required synchronization. lastprivate was also not used because program doesn't need value of last loop iteration after loop finishes

8. Experimental Environment and Compilation Information

Information	Value
CPU model	12th Gen Intel(R) Core(TM) i5-12500H @ 2.50 GHz
Number of cores	12
Number of logical processors	16
RAM	16.0 GB
Operating system	Microsoft Windows 11 Home Single Language, Version 10.0.22621
Compiler	gcc.exe (Rev5, Built by MSYS2 project) 15.1.0
Compilation environment	MSYS2 UCRT64
Compilation command	gcc -O2 -fopenmp image_pipeline_openmp.c -o image_pipeline_openmp.exe

Image size used in main experiments was 4096 x 4096. Each experiment was run 5 times and average execution time was used and program didn't measure external image reading time. Synthetic input image was generated inside program and PGM output files were written after processing. This makes timing focus on image processing pipeline itself

Example run commands:

```
./image_pipeline_openmp.exe 4096 4096 12 static 5
```

```
./image_pipeline_openmp.exe 4096 4096 12 dynamic 5
```

```
./image_pipeline_openmp.exe 4096 4096 12 guided 5
```

9. Performance Metrics

Metrics were used for performance evaluation

- **Execution Time:** average running time of program for related solution
- **Speedup** = sequential time / parallel time
- **Efficiency** = speedup / sumner of threads
- **Scalability:** change in execution time, speedup and efficiency when number of threads increases

Speedup displays number of times parallel solution is faster than sequential solution. Efficiency shows how effectively used threads contribute to speedup and scalability was evaluated in two ways. First input size was increased while number of threads and scheduling type were kept fixed. Second, number of threads was increased for main input size of 4096 x 4096. This made it possible to observe both problem size scalability and thread count scalability

10. Experimental Results

10.1 Input Size Experiments

Input size	Thread	Schedule	Sequentia l Time (s)	Parallel Time (s)	Speedup	Efficiency	Mismatch ed Pixels
1024 x 1024	12	static	0.0062	0.0036	1.7222	0.1435	0
2048 x 2048	12	static	0.0252	0.0094	2.6809	0.2234	0
4096 x 4096	12	static	0.1096	0.0300	3.6533	0.3044	0

Input size experiment speedup increased as image size became larger. For 1024 x 1024 speedup was 1.7222. For 2048 x 2048 it increased to 2.6809. For 4096 x 4096 it reached 3.6533. The results indicate that larger input sizes gain more from parallel execution since amount of work is substantial enough to justify cost of OpenMP overhead. In all input sizes mismatched pixel count was 0

10.2 Thread Number Experiments

Thread	Schedule	Sequential Time (s)	Parallel Time (s)	Speedup	Efficiency	Mismatched Pixels
1	static	0.0996	0.1536	0.6484	0.6484	0
2	static	0.1042	0.0848	1.2288	0.6144	0
4	static	0.1026	0.0606	1.6931	0.4233	0
8	static	0.1048	0.0350	2.9943	0.3743	0
12	static	0.1012	0.0300	3.3733	0.2811	0
16	static	0.1006	0.0286	3.5175	0.2198	0

As the number of threads increased parallel execution time generally decreased. In the 1 thread case parallel version was slower than sequential version. This is expected because parallel version still has OpenMP runtime overhead even when only one thread is used

Best speedup in thread count experiment was obtained with 16 threads. Improvement from 12 to 16 threads was small compared to the earlier increases. This shows that increasing thread count doesn't always produce proportional speedup. After a certain point thread management overhead, cache effects, memory bandwidth limitations and synchronization costs reduce benefit of adding more threads

10.3 Scheduling Experiments

Schedule	Threads	Sequential Time (s)	Parallel Time (s)	Speedup	Efficiency	Mismatched Pixels
static	12	0.1050	0.0320	3.2812	0.2734	0
dynamic	12	0.1014	0.0318	3.1887	0.2657	0
guided	12	0.1034	0.0266	3.8872	0.3239	0

In scheduling experiments number of threads was fixed as 12. Static, dynamic and guided scheduling policies were compared. Main comparison is based on parallel execution time because sequential time was measured separately in each run and may slightly fluctuate due to system load

According to results guided scheduling produced the lowest parallel execution time for this input and system. Static and dynamic scheduling produced close results that shows that scheduling choice can affect performance and should be tested experimentally instead of being assumed

11. Graphs and Interpretations

11.1 Thread Count - Execution Time

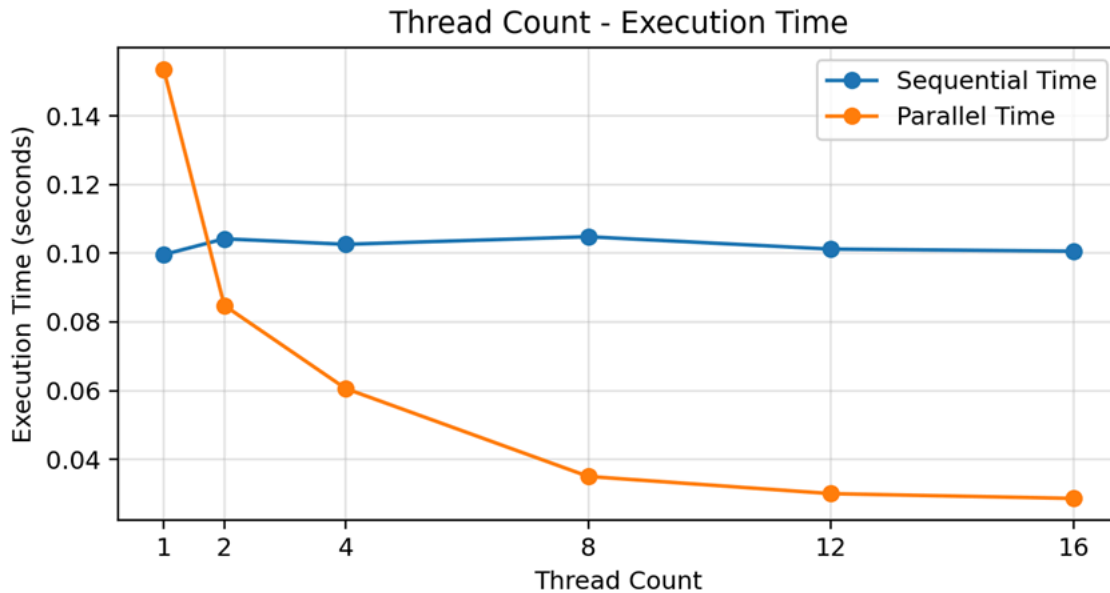


Figure 1. Thread Count - Execution Time

In Figure 1 parallel execution time generally decreased as number of threads increased. 1 thread parallel version was slower than sequential version because it still includes OpenMP runtime overhead. After 8 threads execution time continued to decrease but improvement became smaller and this shows that increasing thread count doesn't always produce proportional performance improvement

11.2 Thread Count - Speedup

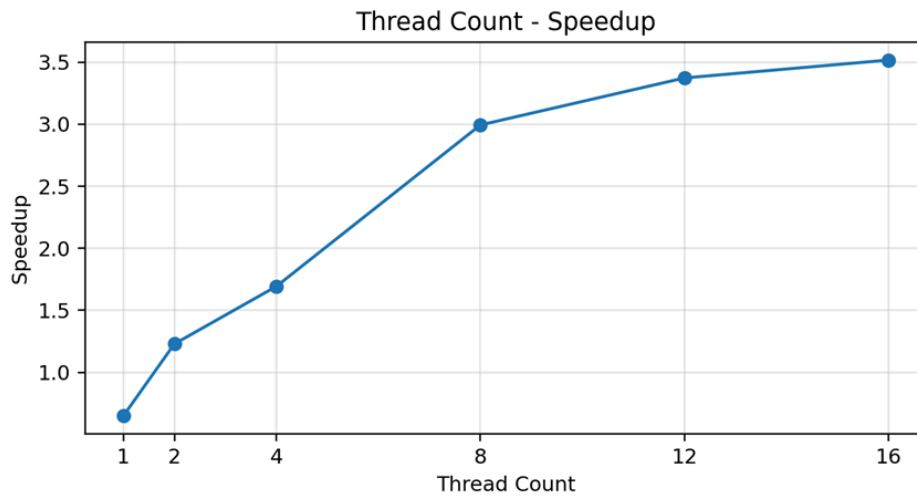


Figure 2. Thread Count- Speedup

In Figure 2 speedup increased as number of threads increased. Best speedup in this experiment was obtained with 16 threads. Increase from 12 to 16 threads was small compared to the earlier increases and this indicates that program benefits from parallelism but scalability becomes limited when number of threads gets close to number of logical processors

11.3 Thread Count - Efficiency

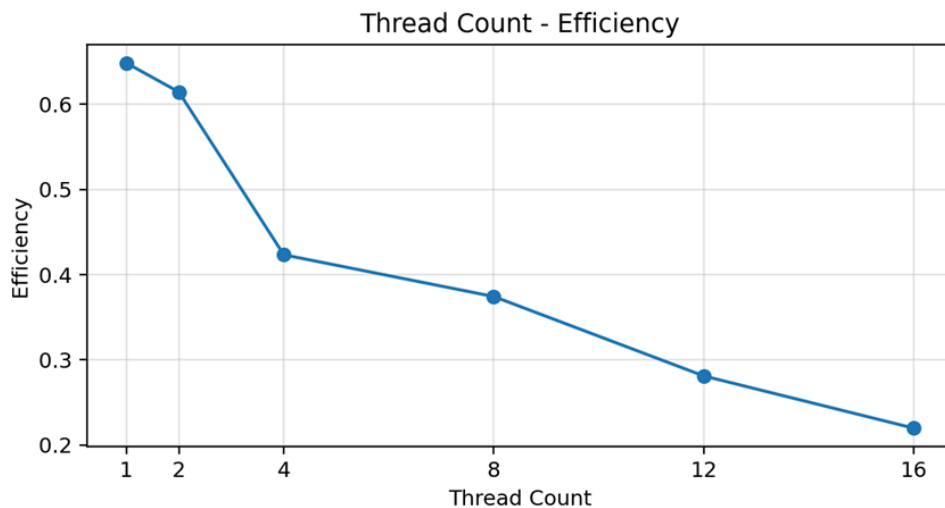


Figure 3. Thread Count- Efficiency

In Figure 3 efficiency decreased as number of threads increased and this is expected in real parallel programs because thread management, scheduling, memory access, cache effects and synchronization costs reduce ideal parallel efficiency. Even though speedup increases each additional thread contributes less performance improvement than previous ones

11.4 Scheduling Type - Execution Time

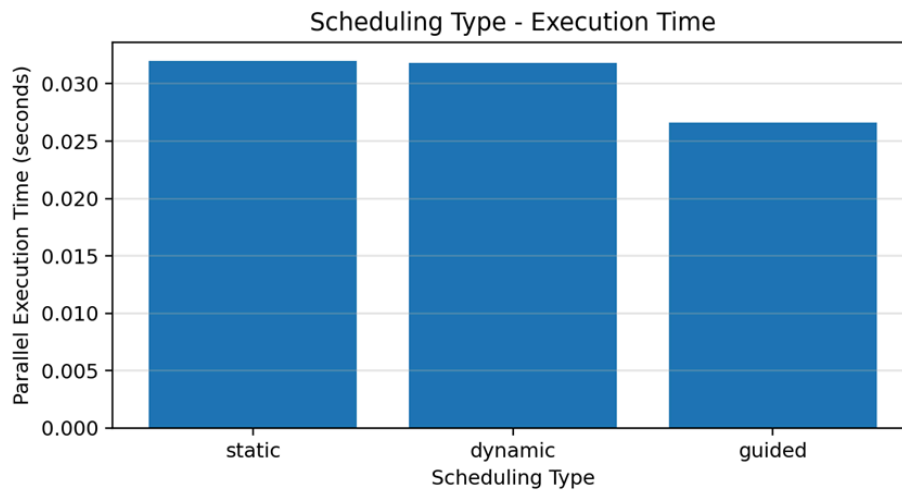


Figure 4. Scheduling Type - Execution Time

In Figure 4 scheduling policy affected parallel execution time. Guided scheduling produced lowest parallel execution time with 0.0266 seconds. Static and dynamic scheduling produced close results but guided scheduling performed better for this input and system. This result shows that scheduling choice should be tested experimentally because its effect may change depending on input size, chunk size, workload balance and hardware characteristics

12. Discussion

Image processing pipeline is appropriate for OpenMP parallelization, according to the results. Main reason is that pixel based operations can be divided among threads. Each thread can process a different part of image and output pixels are written to separate indexes

Input size experiment showed that larger images benefit more from parallel execution. Speedup increased from 1.7222 for 1024 x 1024 to 3.6533 for 4096 x 4096. This is because larger images contain more pixel based work so computation becomes large enough to compensate for OpenMP overhead

Speedup wasn't linear for instance with 16 threads speedup was 3.5175 not close to 16. This is realistic for this type of program because pipeline uses memory heavily and memory bandwidth can become a limiting factor. In addition there are implicit barriers between stages. Barriers are necessary for correctness but they also add synchronization cost

Parallel version with 1 thread was slower than sequential version. This doesn't mean implementation is wrong. It shows overhead of OpenMP and parallel version has a parallel region, scheduling mechanism and reduction handling. When thread count is low this overhead can be larger than benefit

Guided scheduling produced best result in scheduling test. This wasn't assumed before running experiments. Static scheduling is usually good for balanced workloads but in this run guided scheduling produced lowest time. Dynamic scheduling was very close to static in this final run and this shows why scheduling experiments are important

Correctness is also important and in all experiments mismatched pixels were 0 and this means parallel version didn't change output. Use of separate buffers, implicit barriers and reduction helped protect correctness

13. Conclusion

I created OpenMP parallel and sequential image processing pipeline for this project and pipeline consists of thresholding, Sobel edge detection, 3x3 blur and RGB to grayscale conversion. Parallel version uses OpenMP structures such as parallel, for, collapse(2), schedule(runtime), shared, private, firstprivate and reduction

Input sizes of 1024 x 1024, 2048 x 2048, and 4096 x 4096 were used to test the program. A 4096 by 4096 image with 1, 2, 4, 8, 12 and 16 threads was utilized in main thread count experiment. Static, dynamic and guided scheduling were also compared. Each experiment was repeated 5 times and average values were reported

The best speedup in thread count experiment was obtained with 16 threads. In my measurements improvement from 12 to 16 threads was relatively small which shows that scalability becomes limited as number of threads approaches number of logical processors. Input size experiment showed that larger images benefit more from parallel execution because computational workload becomes large enough to compensate for OpenMP overhead

Parallel solution produced correct results in all experiments and mismatched pixel count was always 0. Overall this project demonstrates difference between sequential and parallel execution effect of thread count importance of scheduling and need to analyze data dependency, race condition and synchronization when using OpenMP

14. Compilation and Execution Instructions

Program was compiled and run in MSYS2 UCRT64 terminal and following commands were used:

```
cd /c/Users/WINCHESTER
```

```
gcc -O2 -fopenmp image_pipeline_openmp.c -o image_pipeline_openmp.exe
```

```
./image_pipeline_openmp.exe 4096 4096 12 static 5
```

Program prints measured times, speedup, efficiency, checksum values and mismatched pixel count to terminal. It also prints CSV style line so results can be copied easily into a spreadsheet for table and graph generation. Output images are saved as output_sequential.pgm and output_parallel.pgm

C Source code:

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <omp.h>

#define THRESHOLD 100

#define CHUNK_SIZE 128

static unsigned char clamp_int(int value) {

    if (value < 0) return 0;

    if (value > 255) return 255;

    return (unsigned char)value;

}

static unsigned char *allocate_image(int width, int height) {

    unsigned char *img = (unsigned char *)malloc((size_t)width * height * sizeof(unsigned char));

    if (img == NULL) {

        printf("Memory allocation failed.\n");

        exit(1);

    }

    return img;

}

static void generate_rgb_image(unsigned char *r, unsigned char *g, unsigned char *b,

                               int width, int height) {

    for (int y = 0; y < height; y++) {

        for (int x = 0; x < width; x++) {

            int idx = y * width + x;

            r[idx] = (unsigned char)((x * 255) / width);

            g[idx] = (unsigned char)((y * 255) / height);

            b[idx] = (unsigned char)((((x / 32) + (y / 32)) % 2) * 255);

        }

    }

}
```

```

    }
}

static int write_pgm(const char *filename, unsigned char *image, int width, int height) {
    FILE *file = fopen(filename, "wb");

    if (file == NULL) {
        printf("Could not open output file: %s\n", filename);
        return 0;
    }

    fprintf(file, "P5\n%d %d\n255\n", width, height);
    fwrite(image, sizeof(unsigned char), (size_t)width * height, file);
    fclose(file);

    return 1;
}

static void set_schedule_policy(const char *schedule_name) {
    if (strcmp(schedule_name, "static") == 0) {
        omp_set_schedule(omp_sched_static, CHUNK_SIZE);
    } else if (strcmp(schedule_name, "dynamic") == 0) {
        omp_set_schedule(omp_sched_dynamic, CHUNK_SIZE);
    } else if (strcmp(schedule_name, "guided") == 0) {
        omp_set_schedule(omp_sched_guided, CHUNK_SIZE);
    } else {
        printf("Unknown schedule type. Use: static, dynamic, or guided.\n");
        exit(1);
    }
}

static long long sequential_pipeline(unsigned char *r, unsigned char *g, unsigned char *b,
                                     unsigned char *gray, unsigned char *blur,
                                     unsigned char *edge, unsigned char *out,
                                     int width, int height, int threshold) {

    long long checksum = 0;

```

```

/*
    Stage 1: RGB to grayscale

*/

for (int y = 0; y < height; y++) {
    for (int x = 0; x < width; x++) {
        int idx = y * width + x;

        gray[idx] = (unsigned char)((77 * r[idx] + 150 * g[idx] + 29 * b[idx]) / 256);
    }
}

/*
    Stage 2: 3x3 blur
*/

for (int y = 0; y < height; y++) {
    for (int x = 0; x < width; x++) {
        int idx = y * width + x;

        if (x == 0 || y == 0 || x == width - 1 || y == height - 1) {
            blur[idx] = gray[idx];
        } else {
            int sum = 0;

            for (int ky = -1; ky <= 1; ky++) {
                for (int kx = -1; kx <= 1; kx++) {
                    sum += gray[(y + ky) * width + (x + kx)];
                }
            }

            blur[idx] = (unsigned char)(sum / 9);
        }
    }
}

/*
    Stage 3: Sobel edge detection
*/

```

```

for (int y = 0; y < height; y++) {
    for (int x = 0; x < width; x++) {
        int idx = y * width + x;

        if (x == 0 || y == 0 || x == width - 1 || y == height - 1) {
            edge[idx] = 0;
        } else {
            int gx = 0;
            int gy = 0;

            gx += -1 * blur[(y - 1) * width + (x - 1)];
            gx += 1 * blur[(y - 1) * width + (x + 1)];
            gx += -2 * blur[y * width + (x - 1)];
            gx += 2 * blur[y * width + (x + 1)];
            gx += -1 * blur[(y + 1) * width + (x - 1)];
            gx += 1 * blur[(y + 1) * width + (x + 1)];

            gy += -1 * blur[(y - 1) * width + (x - 1)];
            gy += -2 * blur[(y - 1) * width + x];
            gy += -1 * blur[(y - 1) * width + (x + 1)];
            gy += 1 * blur[(y + 1) * width + (x - 1)];
            gy += 2 * blur[(y + 1) * width + x];
            gy += 1 * blur[(y + 1) * width + (x + 1)];

            edge[idx] = clamp_int(abs(gx) + abs(gy));
        }
    }
}

for (int y = 0; y < height; y++) {
    for (int x = 0; x < width; x++) {
        int idx = y * width + x;

        out[idx] = (edge[idx] > threshold) ? 255 : 0;
        checksum += out[idx];
    }
}

```

```

    }

    return checksum;
}

static long long parallel_pipeline(unsigned char *r, unsigned char *g, unsigned char *b,
                                   unsigned char *gray, unsigned char *blur,
                                   unsigned char *edge, unsigned char *out,
                                   int width, int height, int threshold) {

    long long checksum = 0;

    int x, y;

    #pragma omp parallel default(none) shared(r, g, b, gray, blur, edge, out) firstprivate(width,
height, threshold) private(x, y) reduction(+:checksum)

    {
        /*
         * Stage 1: RGB to grayscale
         */
        #pragma omp for collapse(2) schedule(runtime)
        for (y = 0; y < height; y++) {
            for (x = 0; x < width; x++) {
                int idx = y * width + x;

                gray[idx] = (unsigned char)((77 * r[idx] + 150 * g[idx] + 29 * b[idx]) / 256);
            }
        }

        /*
         * Stage 2: 3x3 blur
         */
        #pragma omp for collapse(2) schedule(runtime)
        for (y = 0; y < height; y++) {
            for (x = 0; x < width; x++) {

                int idx = y * width + x;

                if (x == 0 || y == 0 || x == width - 1 || y == height - 1) {

                    blur[idx] = gray[idx];

```

```

        } else {

            int sum = 0;

            for (int ky = -1; ky <= 1; ky++) {
                for (int kx = -1; kx <= 1; kx++) {
                    sum += gray[(y + ky) * width + (x + kx)];
                }
            }

            blur[idx] = (unsigned char) (sum / 9);
        }
    }
}

/*
    Stage 3: Sobel edge detection
*/

#pragma omp for collapse(2) schedule(runtime)
for (y = 0; y < height; y++) {
    for (x = 0; x < width; x++) {
        int idx = y * width + x;

        if (x == 0 || y == 0 || x == width - 1 || y == height - 1) {
            edge[idx] = 0;
        } else {
            int gx = 0;
            int gy = 0;

            gx += -1 * blur[(y - 1) * width + (x - 1)];
            gx +=  1 * blur[(y - 1) * width + (x + 1)];
            gx += -2 * blur[y * width + (x - 1)];
            gx +=  2 * blur[y * width + (x + 1)];
            gx += -1 * blur[(y + 1) * width + (x - 1)];
            gx +=  1 * blur[(y + 1) * width + (x + 1)];

            gy += -1 * blur[(y - 1) * width + (x - 1)];
            gy += -2 * blur[(y - 1) * width + x];

```

```

        gy += -1 * blur[(y - 1) * width + (x + 1)];

        gy += 1 * blur[(y + 1) * width + (x - 1)];

        gy += 2 * blur[(y + 1) * width + x];

        gy += 1 * blur[(y + 1) * width + (x + 1)];

        edge[idx] = clamp_int(abs(gx) + abs(gy));

    }

}

/*
   Stage 4: Thresholding and checksum calculation
*/

#pragma omp for collapse(2) schedule(runtime)
for (y = 0; y < height; y++) {
    for (x = 0; x < width; x++) {
        int idx = y * width + x;

        out[idx] = (edge[idx] > threshold) ? 255 : 0;

        checksum += out[idx];
    }
}

return checksum;
}

static long long count_differences(unsigned char *a, unsigned char *b,
                                   int width, int height) {

    long long diff_count = 0;

    size_t total = (size_t)width * height;

    for (size_t i = 0; i < total; i++) {
        if (a[i] != b[i]) {
            diff_count++;
        }
    }
}

```



```

    return diff_count;
}

int main(int argc, char *argv[]) {
    int width = 2048;
    int height = 2048;
    int threads = 4;
    int runs = 5;
    const char *schedule_name = "static";

    if (argc >= 3) {
        width = atoi(argv[1]);
        height = atoi(argv[2]);
    }

    if (argc >= 4) {
        threads = atoi(argv[3]);
    }

    if (argc >= 5) {
        schedule_name = argv[4];
    }

    if (argc >= 6) {
        runs = atoi(argv[5]);
    }

    if (width < 3 || height < 3 || threads < 1 || runs < 1) {
        printf("Usage: %s [width] [height] [threads] [static|dynamic|guided] [runs]\n", argv[0]);
        return 1;
    }

    omp_set_num_threads(threads);
    set_schedule_policy(schedule_name);

    unsigned char *r = allocate_image(width, height);

```

```
unsigned char *g = allocate_image(width, height);

unsigned char *b = allocate_image(width, height);


unsigned char *gray_seq = allocate_image(width, height);
unsigned char *blur_seq = allocate_image(width, height);
unsigned char *edge_seq = allocate_image(width, height);
unsigned char *out_seq = allocate_image(width, height);


unsigned char *gray_par = allocate_image(width, height);
unsigned char *blur_par = allocate_image(width, height);
unsigned char *edge_par = allocate_image(width, height);
unsigned char *out_par = allocate_image(width, height);


generate_rgb_image(r, g, b, width, height);


double seq_total_time = 0.0;
double par_total_time = 0.0;


long long seq_checksum = 0;
long long par_checksum = 0;


for (int i = 0; i < runs; i++) {

    double start = omp_get_wtime();


    seq_checksum = sequential_pipeline(r, g, b,

                                     gray_seq, blur_seq, edge_seq, out_seq,

                                     width, height, THRESHOLD);


    double end = omp_get_wtime();

    seq_total_time += end - start;

}


for (int i = 0; i < runs; i++) {

    double start = omp_get_wtime();


    par_checksum = parallel_pipeline(r, g, b,
```

```

        gray_par, blur_par, edge_par, out_par,
        width, height, THRESHOLD);

    double end = omp_get_wtime();
    par_total_time += end - start;
}

double seq_avg = seq_total_time / runs;
double par_avg = par_total_time / runs;
double speedup = seq_avg / par_avg;
double efficiency = speedup / threads;

long long differences = count_differences(out_seq, out_par, width, height);

write_pgm("output_sequential.pgm", out_seq, width, height);
write_pgm("output_parallel.pgm", out_par, width, height);

printf("\nOpenMP Image Processing Pipeline\n");
printf("Image size           : %d x %d\n", width, height);
printf("Threads              : %d\n", threads);
printf("Schedule                : %s, chunk=%d\n", schedule_name, CHUNK_SIZE);
printf("Runs                    : %d\n", runs);
printf("Sequential avg time (s) : %.6f\n", seq_avg);
printf("Parallel avg time (s)   : %.6f\n", par_avg);
printf("Speedup                 : %.4f\n", speedup);
printf("Efficiency              : %.4f\n", efficiency);
printf("Sequential checksum     : %lld\n", seq_checksum);
printf("Parallel checksum       : %lld\n", par_checksum);
printf("Mismatched pixels       : %lld\n", differences);
printf("Output files            : output_sequential.pgm, output_parallel.pgm\n");

printf("CSV,%d,%d,%d,%s,%d,%.6f,%.6f,%.4f,%.4f,%lld,%lld,%lld\n",
    width, height, threads, schedule_name, runs,
    seq_avg, par_avg, speedup, efficiency,
    seq_checksum, par_checksum, differences);

free(r);

```

```
    free(g);

    free(b);


    free(gray_seq);
    free(blur_seq);
    free(edge_seq);
    free(out_seq);


    free(gray_par);
    free(blur_par);
    free(edge_par);
    free(out_par);


    return 0;
}
```